

Java Generics - Cheat Sheet

Generics let you write code that works with any type while keeping compile-time type safety. Before Java 5, you used `Object` everywhere and cast at runtime. One wrong cast and the JVM crashed at the worst moment. Generics moved those checks to compile time.

The catch: Java added generics via type erasure, which means the runtime barely knows generics exist. This design preserved backward compatibility with pre-5 code but creates a long list of corner cases that confuse devs forever.

Why generics

Before:

```
List names = new ArrayList();
names.add("alice");
names.add(42); // compiles fine
String first = (String) names.get(0); // works
String second = (String) names.get(1); // ClassCastException at runtime
```

After:

```
List<String> names = new ArrayList<>();
names.add("alice");
names.add(42); // compile error
```

Same data structure, but the compiler stops the bug before it ships.

The benefits:

- Type safety at compile time. The compiler catches type mismatches.
- No casts in user code. Cleaner reads.
- Reusable abstractions. Write one `List<T>` instead of `IntList`, `StringList`, etc.

Generic classes

A class can declare type parameters in angle brackets.

```

public class Box<T> {
    private T value;
    public Box(T value) {
        this.value = value;
    }
    public T get() {
        return value;
    }
    public void set(T value) {
        this.value = value;
    }
}

// Usage
Box<String> stringBox = new Box<>("hello");
Box<Integer> intBox = new Box<>(42);
String s = stringBox.get(); // no cast needed

```

Multiple type parameters:

```

public class Pair<K, V> {
    private final K key;
    private final V value;
    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }
    public K key() {
        return key;
    }
    public V value() {
        return value;
    }
}

Pair<String, Integer> p = new Pair<>("age", 30);

```

Conventions for parameter names:

- T for Type
- E for Element (in collections)
- K , V for Key, Value (in maps)
- R for Return type
- N for Number

These are conventions, not language rules. Pick names that read well.

Generic methods

A method can declare its own type parameters, independent of the class.

```

public class Util {
    public static <T> T firstOf(List<T> list) {
        return list.get(0);
    }

    public static <T> List<T> repeat(T value, int times) {
        List<T> result = new ArrayList<>();
        for (int i = 0; i < times; i++) {
            result.add(value);
        }
        return result;
    }
}

String s = Util.firstOf(List.of("a", "b", "c"));
List<Integer> threes = Util.repeat(3, 5);

```

The `<T>` before the return type declares the type parameter. The compiler infers it from the arguments.

Sometimes you need to specify the type explicitly:

```
List<String> empty = Util.<String>repeat("", 0);
```

Rare in practice.

Bounded type parameters

Restrict the type to a subtype of something.

```

public <T extends Number> double sum(List<T> list) {
    double total = 0;
    for (T n : list) {
        total += n.doubleValue();
    }
    return total;
}

```

Now `T` must be `Number` or a subtype (`Integer`, `Double`, etc.). You get `doubleValue()` for free.

Multiple bounds with `&`:

```

public <T extends Comparable<T> & Serializable> T max(List<T> list) {
    // T must implement both Comparable and Serializable
}

```

Only one class is allowed in the bounds. The rest must be interfaces. The class (if present) must come first.

`extends` here means “extends or implements”. Same keyword, both directions.

Wildcards

Wildcards (`?`) let you write methods that accept a range of generic types.

Upper bounded: `? extends T`

“Some unknown type that is a subtype of T.”

```
public double sumNumbers(List<? extends Number> list) {
    double total = 0;
    for (Number n : list) {
        total += n.doubleValue();
    }
    return total;
}

sumNumbers(List.of(1, 2, 3));           // List<Integer> works
sumNumbers(List.of(1.0, 2.0, 3.0));    // List<Double> works
```

You can read from the list (as `Number`). You cannot add to it. The compiler doesn’t know which subtype the list actually holds.

Lower bounded: `? super T`

“Some unknown type that is a supertype of T.”

```
public void addIntegers(List<? super Integer> list) {
    list.add(1);
    list.add(2);
    list.add(3);
}

List<Number> numbers = new ArrayList<>();
addIntegers(numbers); // works
List<Object> objects = new ArrayList<>();
addIntegers(objects); // also works
```

You can add `Integers` (any supertype will accept them). You can only read them as `Object` .

Unbounded: `?`

“Some unknown type.”

```
public int sizeOf(List<?> list) {
    return list.size();
}
```

You can read elements as `Object` and call methods that don't depend on the type. Mainly useful for read-only operations that don't care about the element type.

PECS (Producer Extends, Consumer Super)

The mnemonic that captures wildcard usage. Coined by Joshua Bloch in Effective Java.

- If the parameter produces values for the method to use, use `? extends T`.
- If the parameter consumes values from the method, use `? super T`.
- If it does both, use exactly `T`.

Example:

```
public static <T> void copy(List<? extends T> source, List<? super T> dest) {  
    for (T item : source) {  
        dest.add(item);  
    }  
}
```

`source` produces `T` values. `dest` consumes them. Both wildcards make the method work across a range of caller types.

Used everywhere in `java.util.Collections`:

```
public static <T> void sort(List<T> list, Comparator<? super T> c) { ... }  
public static <T extends Comparable<? super T>> void sort(List<T> list) { ... }
```

Type erasure

The single most important concept for understanding Java generics.

At runtime, the JVM doesn't know about generic type parameters. The compiler uses them for checking, then erases them.

```
List<String> a = new ArrayList<>();  
List<Integer> b = new ArrayList<>();  
a.getClass() == b.getClass(); // true
```

Both `a` and `b` are just `ArrayList` at runtime. The `<String>` and `<Integer>` are compile-time only.

What erasure does:

- Replaces type parameters with their bounds (or `Object` if unbounded).
- Inserts casts at use sites.
- Generates bridge methods to preserve polymorphism.

Example. Source:

```
public class Box<T> {  
    private T value;  
    public T get() { return value; }  
}
```

After erasure:

```
public class Box {  
    private Object value;  
    public Object get() { return value; }  
}
```

And the call site:

```
Box<String> b = new Box<>("hi");  
String s = b.get(); // compiler inserts (String) cast
```

This is why most of the limitations below exist.

Generic limitations

Things you can't do because of type erasure. Memorize these. They show up in interviews.

1. Cannot use primitives as type parameters

```
List<int> list = ...; // compile error  
List<Integer> list = ...; // works
```

Use boxed types. Project Valhalla (value classes) will change this eventually.

2. Cannot create instances of type parameters

```
public <T> T create() {  
    return new T(); // compile error  
}
```

The runtime doesn't know what `T` is. Workaround, pass a `Class<T>` and use reflection:

```
public <T> T create(Class<T> clazz) throws Exception {  
    return clazz.getDeclaredConstructor().newInstance();  
}
```

Or pass a `Supplier<T>` :

```
public <T> T create(Supplier<T> factory) {  
    return factory.get();  
}
```

3. Cannot create generic arrays

```
List<String>[] arr = new List<String>[10]; // compile error  
List<?>[] arr = new List<?>[10];         // legal but loosely typed
```

Workaround, use `ArrayList<List<String>>` , or cast from a raw array with `@SuppressWarnings("unchecked")` .

4. Cannot have static fields of type parameter

```
public class Box<T> {  
    private static T value; // compile error  
}
```

Static fields belong to the class. Each instance can have a different `T` , so a single static `T` has no meaningful value.

5. Cannot catch generic exceptions

```
try { ... }  
catch (T e) { ... } // compile error
```

The runtime needs to match exception classes. Type erasure breaks this.

6. Cannot use instanceof with parameterized types

```
if (obj instanceof List<String>) // compile error  
if (obj instanceof List<?>)      // legal  
if (obj instanceof List)         // legal (raw)
```

At runtime, all `List<X>` look identical.

7. Cannot overload by erased type

```
public void process(List<String> list) { ... }  
public void process(List<Integer> list) { ... } // compile error
```

Both methods erase to the same signature: `process(List)` .

8. Cannot create parameterized arrays of inner classes

Mostly historical. Rarely matters in modern code.

Bridge methods

The compiler generates these to preserve polymorphism after erasure.

Consider:

```
public class Node<T> {
    public T data;
    public void setData(T data) {
        this.data = data;
    }
}

public class MyNode extends Node<Integer> {
    @Override
    public void setData(Integer data) {
        // ...
    }
}
```

After erasure, `Node` has `setData(Object)`. `MyNode` has `setData(Integer)`. These don't override naturally.

The compiler adds a bridge method to `MyNode` :

```
public void setData(Object data) {
    setData((Integer) data); // calls the real method
}
```

You won't write bridge methods, but you'll see them in stack traces and reflection output. Now you know what they are.

Reifiable vs non-reifiable types

A type is reifiable if its complete type information is available at runtime. Examples:

- Primitives (`int`)
- Non-generic classes (`String`)
- Raw types (`List`)
- Wildcard with no bound (`List<?>`)

A type is non-reifiable if some of it gets erased. Examples:

- `List<String>`

- `List<? extends Number>`
- `T` (a type parameter)

You can use `instanceof` and create arrays only with reifiable types. That's why `instanceof List<String>` is illegal but `instanceof List<?>` is fine.

Type inference and the diamond operator

Since Java 7:

```
Map<String, List<Integer>> m = new HashMap<String, List<Integer>>(); // verbose
Map<String, List<Integer>> m = new HashMap<>();                      // diamond
```

The compiler infers the type from the left side.

Java 8+ extended inference to method arguments:

```
List<String> empty = Collections.emptyList();
// Compiler infers <String> from the assignment context
```

Java 10+ added `var` for local variables:

```
var users = new ArrayList<User>(); // type inferred as ArrayList<User>
```

`var` works for locals only. You still need to declare types for fields, parameters, and return types.

Generic patterns in the wild

Comparator

```
Comparator<User> byName = Comparator.comparing(User::getName);
Comparator<User> byAge = Comparator.comparingInt(User::getAge);
users.sort(byName.thenComparing(byAge));
```

CompletableFuture

```
CompletableFuture<User> userFuture =
    CompletableFuture.supplyAsync(() -> fetchUser(id));
CompletableFuture<String> nameFuture =
    userFuture.thenApply(User::getName);
```

Stream

```
List<String> names = users.stream()
    .filter(u -> u.isActive())
    .map(User::getName)
    .collect(Collectors.toList());
```

Builder (recursive bound)

```
public class Request<T extends Request<T>> {
    @SuppressWarnings("unchecked")
    public T withHeader(String key, String value) {
        // ...
        return (T) this;
    }
}

public class HttpRequest extends Request<HttpRequest> {
    // inherits withHeader returning HttpRequest, not Request
}
```

The `T extends Request<T>` is the “curiously recurring template pattern” (CRTP). It lets the parent method return the child type. Common in fluent builders.

Best practices

- Always parameterize generic types. No raw `List`. Use `List<String>` or at least `List<?>`.
- Use bounded wildcards for flexibility. Apply PECS so methods accept a range of types.
- Use generic methods for static utilities. They infer well and stay reusable.
- Avoid wildcards in return types. They push complexity onto every caller. Return concrete `List<T>`.
- Don’t suppress warnings without thinking. `@SuppressWarnings("unchecked")` should come with a comment explaining why it’s safe.
- Prefer generic classes over raw types. Raw types exist only for backward compat with pre-5 code.
- Use the diamond operator. Cleaner than repeating the type parameters.

Common gotchas

- `List<Object>` is NOT a supertype of `List<String>`. Generics are invariant. Use `List<? extends Object>` if you need that relationship, or just `List<?>`.
- Arrays and generics don’t mix. Arrays are covariant and reified. Generics are invariant and erased. Mixing them leads to `ArrayStoreException` at runtime.
- `new T[]` is illegal. Type erasure makes the array creation impossible. Use `ArrayList<T>` or pass a `Class<T>` and use `Array.newInstance`.

- Heap pollution. When generic and raw types mix, you can store the wrong type into a parameterized collection. The compiler warns, but it compiles. Don't ignore the warning.
- `Class<T>` vs `Class<?>`. `Class<String>` is specific. `Class<?>` is "any class". `Class` (raw) is legacy. Pick the right one.
- `Optional<T>` belongs on return types for methods that may not find a value. Avoid it as a field type or a method parameter.
- `equals` and generics. The `equals` method takes `Object`. Don't accidentally write `boolean equals(MyClass other)`. The compiler won't warn you, and the method won't override.
- Bridge methods in stack traces. A `NullPointerException` in a "synthetic" method is usually a bridge method. The real bug is in the actual overridden method.
- Reflection on generic types. `field.getGenericType()` gives you the parameterized type at runtime. Useful for ORMs and serializers. Doesn't mean erasure is gone.
- Static factory methods need explicit type parameters when inference fails. `Collections.<String>emptyList()` when the context can't infer.

Quick reference

Concept	Key fact
Generic class	<code>class Box<T> { T value; }</code>
Generic method	<code><T> T firstOf(List<T> list)</code>
Bounded type	<code><T extends Number></code>
Multiple bounds	<code><T extends Number & Comparable<T>></code>
Upper wildcard	<code>List<? extends Number></code> (producer)
Lower wildcard	<code>List<? super Integer></code> (consumer)
Unbounded wildcard	<code>List<?></code> (any list)
PECS	Producer Extends, Consumer Super
Type erasure	Generic type info gone at runtime
Raw type	<code>List</code> (legacy, avoid)
Diamond	<code>new ArrayList<>()</code>
Reifiable	Type known at runtime
Bridge method	Compiler-generated to preserve polymorphism
Heap pollution	Raw types mixing with parameterized ones